Лабораторна робота №4

Тема: AOP

(Aspect Oriented Programming)

Як розробник, що розширює цю кодову базу, я хочу, щоб наскрізна функціональність (наприклад, логування, аудит, контроль доступу чи вимірювання часу виконання) була реалізована у вигляді пари «власна анотація + аспект». Це дозволить застосовувати відповідну поведінку декларативно там, де це необхідно, замість того щоб вручну копіювати код у кожен метод, що його потребує.

Опис:

Створити 3 **власні анотації** та 3 **відповідні аспекти** із Advice типом на власний розсуд. Обрати реальну наскрізну функціональність (cross-cutting concern), а не просто перейменовану копію аспекту для логування чи прикладу контролю доступу з лекції. Застосувати цю анотацію принаймні до одного методу у робочому коді.

Завдання складається з обов’язкових частин:

1. Сам аспект, правильно налаштований за допомогою pointcut, що виконує дію, яку можна реально відстежити (запис у лог, викидання винятку, зміна значення, що повертається) залежно від обраної функціональності.
2. Навмисна демонстрація проблеми самовиклику (self-invocation): той самий анотований метод викликається двома різними способами, причому один шлях виклику спрацьовує, а інший не спрацьовує без жодних повідомлень про помилку.

Сценарій 1: Власна анотація правильно пов’язана з аспектом

GIVEN: власна анотація з @Retention(RUNTIME) та відповідним @Target

WHEN: pointcut аспекту використовує @annotation(...) для її виявлення

THEN: застосування анотації до методу призводить до виконання advice аспекту під час виклику цього методу ззовні його класу

Сценарій: Наскрізна поведінка аспекту дійсно спрацьовує

GIVEN: метод, позначений власною анотацією

WHEN: він викликається ззовні, і виклик доходить до проксі-об'єкта

THEN: поведінка advice стає помітною. Наприклад, з’являється запис у лозі, виникає виняток, змінюється значення, що повертається, тощо

Сценарій: Зовнішній виклик правильно активує аспект

GIVEN: метод, позначений власною анотацією

WHEN: він викликається з іншого біна (або безпосередньо з тесту)

THEN: advice аспекту спрацьовує саме так, як описано вище

Сценарій: Самостійний виклик (self-invocation) НЕ активує аспект

GIVEN: той самий анотований метод

WHEN: він викликається через «this.method()» (або як некваліфікований виклик) з іншого методу того самого класу

THEN: advice аспекту НЕ спрацьовує і ця відсутність спрацювання є свідомим задумом вправи, а не помилкою, яку слід виправити перед здачею лабораторної роботи.

Сценарій: Advice перевіряється модульним тестом (unit test) безпосередньо

GIVEN: клас аспекту

WHEN: він проходить тестування

THEN: для ProceedingJoinPoint створюється mock-об'єкт - без контексту Spring і без @SpringBootTest

AND: тест перевіряє фактичну логіку прийняття рішень усередині advice

(наприклад, чи викликається proceed(), або що відбувається, якщо ні).